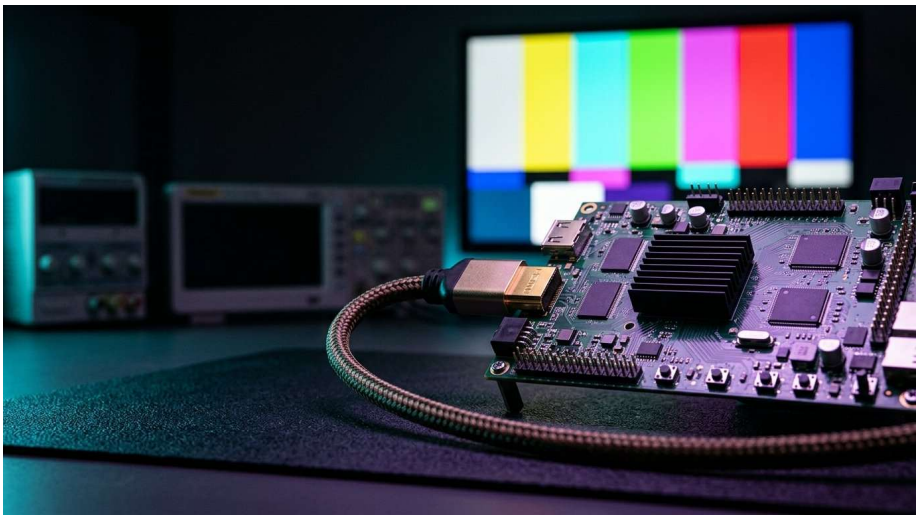
DIGITAL SYSTEMS LAB

HDMI CONTROLLER AND DISPLAY DRIVER

Lab 03



OBJECTIVE

Design and implementation of an FPGA-based HDMI controller capable of displaying a 640 × 480 image at 60 Hz, using VRAM, video synchronization signals, TMDS encoding, and serial transmission.

ANASTASIA EFTHYMIOU – ANDROMACHE SPYROU

03988 – 04059

19 December 2025

Contents

OBJECTIVE 1
SUMMARY & INTRODUCTION 3
Summary 3
Introduction 3
PART A: VRAM Implementation 4
A.1 Implementation 4
RGB Color Encoding 5
A.2 Verification 5
PART B: HSYNC, VSYNC, and ACTIVEDRAW Signals 6
B.1 Implementation 6
B.2 Verification 7
PART C: TMDS Encoding 8
C.1 Implementation 8
C.2 Verification 8
PART D: HDMI Controller Integration 10
D.1 Implementation 10
D.2 Verification 10
D.3 Experiment and Final Implementation 11
CONCLUSIONS 11

SUMMARY & INTRODUCTION

Summary

In this project, an HDMI controller/driver was implemented to drive a conventional display and render an image on it. The display operates at a resolution of 640 × 480 pixels and a refresh rate of 60 Hz, while the image is stored in a 128 × 96 video RAM (VRAM) comprising three internal block RAMs (BRAMs), one for each of the red, green, and blue (RGB) color channels. HDMI frame timing includes an active-drawing interval during which color data are transmitted, as well as horizontal and vertical synchronization, front-porch, and back-porch intervals during which no pixels are displayed. In addition, Transition-Minimized Differential Signaling (TMDS) encoding is used to reduce noise and minimize transitions between logic 0 and logic 1, producing 10-bit output symbols. The resulting symbols are then transmitted serially over the HDMI link. The system was tested and verified through simulations and practical implementation on an FPGA.

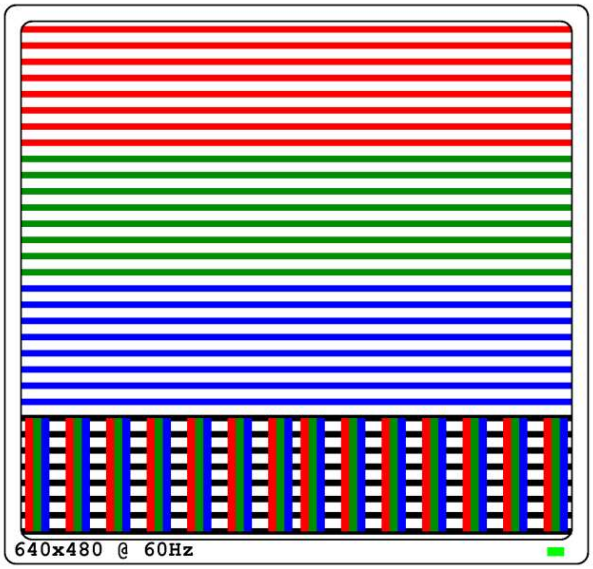
Introduction

The aim of this assignment was to implement an HDMI controller using five states to manage pixel timing and transmit red, green, and blue color data. Due to the chosen memory organization, VRAM stores an image of 128 × 96 pixels, with each dimension reduced by a factor of five compared with the 640 × 480 display resolution. Frame timing was implemented using horizontal and vertical counters to drive the activedraw, hsync, and vsync signals. TMDS encoding was also required to reduce noise and frequent signal transitions through XOR and XNOR operations and analysis of the input bit sequence. Finally, the modules were connected to serialize the encoded bits onto the HDMI link, while each pixel address was mapped to its corresponding color.

PART A: VRAM Implementation

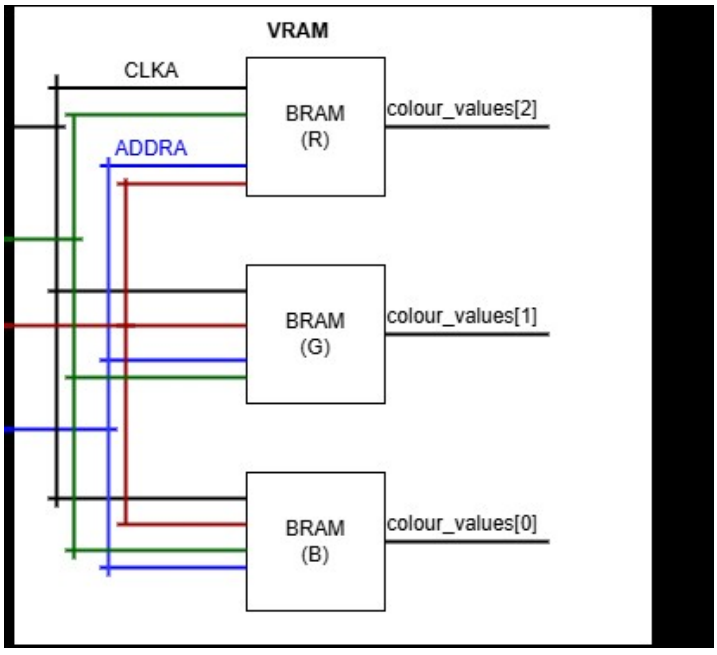
A.1 Implementation

The objective is to display the following image:



Target test pattern for the 640 × 480 display.

To represent the image at 128 × 96 resolution, the number of rows assigned to each color pattern was calculated. The image contains 128 × 96 = 12,288 pixels. With four distinct color patterns, each occupies 12,288 / 4 = 3,072 pixels, corresponding to 3,072 / 128 = 24 rows of 128 pixels each.



VRAM architecture with independent red, green, and blue BRAM channels.

In the first part of the assignment, a VRAM (Video RAM) module was created as a core component of the HDMI controller, since its contents are continuously displayed on the screen. The VRAM has a resolution of 128 × 96 pixels and is implemented using three BRAMs, one for each color channel.

The VRAM module accepts the address of an image pixel. Based on the initialization contents of the BRAMs, it outputs three bits that determine the pixel color according to the red, green, and blue (RGB) values.

RGB Color Encoding

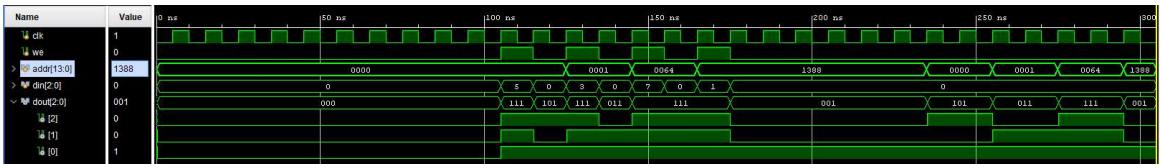
Red (R)	Green (G)	Blue (B)	Resulting Color
0	0	0	Black
0	0	1	Blue
0	1	0	Green
0	1	1	Cyan
1	0	0	Red
1	0	1	Magenta
1	1	0	Yellow
1	1	1	White

RGB combinations generated by the three BRAM output bits (recreated in English).

A.2 Verification

To simulate the VRAM, a testbench named VRAM_tb was implemented with a 10 ns clock period for convenience. The pixel_address vector was updated at suitable intervals with different values to thoroughly verify the behavior of the circuit. Specifically, the following cases were checked:

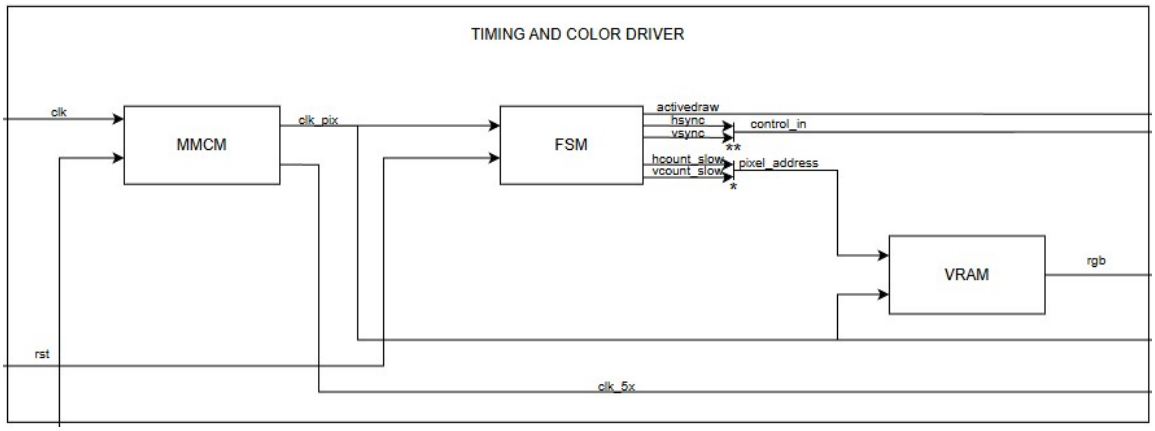
- For pixel_address = 000000000000001 (first row, second pixel), rgb correctly takes the value 100, since the first row of the initialized image contains only red.
- For pixel_address = 001100000000001 (24th row, second pixel), rgb becomes 010; this row marks the beginning of the green region, so only the r bit has a value of 1.
- For pixel_address = 011000000000000 (48th row, first pixel), rgb takes the value 001, since this row contains only blue.



VRAM testbench waveform showing address changes and RGB output.

PART B: HSYNC, VSYNC, and ACTIVEDRAW Signals

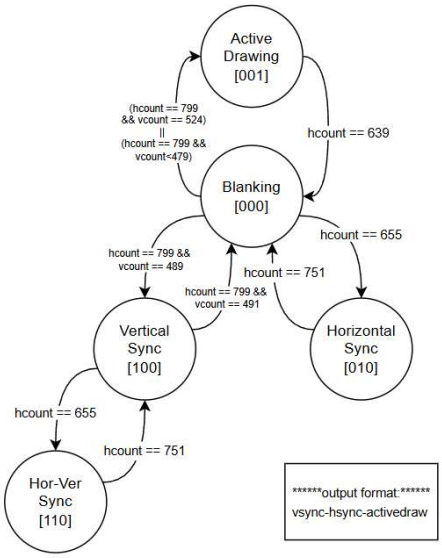
B.1 Implementation



Timing and color driver incorporating MMCM, FSM, and VRAM.

In this part, one counter was implemented for the horizontal dimension (pixels per row) and another for the vertical dimension (total number of rows) of the 640 × 480, 60 Hz image. Two additional counters were used, incrementing once for every five increments of the original counters, because the 128 × 96 stored image has one-fifth the dimensions of the 640 × 480 display.

A five-state Moore finite-state machine (FSM) was implemented according to the regions indicated by the counter values:



Five-state Moore FSM for active video and synchronization intervals.

- ActiveDrawing: Pixel addresses are available when the counters are in the ranges 0–639 and 0–479, and the activedraw signal is asserted. When the horizontal counter reaches 639, the FSM transitions to Blanking.
- Blanking: This state encompasses the front porch and back porch, during which pixel addresses are unavailable. The counter values determine whether the next state is horizontal synchronization (HorSync), vertical synchronization (VertSync), or active drawing (ActiveDrawing).
- HorSync: The horizontal counter takes values from 655 to 751 and hsync is asserted (1). The next state is Blanking, corresponding to the horizontal back porch.

- VertSync: The vertical counter takes values 489 and 490 while the horizontal counter has exceeded 799, and vsync is asserted. The next state is either Blanking (vertical back porch) or HorVertSync, where hsync and vsync overlap.
- HorVertSync: Both hsync and vsync are asserted (1), while the horizontal and vertical counters take the values 655–751 and 489–490, respectively. The next state is VertSync.

Overall, the module contains two always blocks for the FSM (one sequential and one combinational), along with four additional sequential always blocks for assigning and updating the counters.

The top-level module is timing_and_control_driver. Its MMCM reduces the board clock frequency from 100 MHz to 25.175 MHz.

The selected parameters were clkfbout_mult_f = 17.625, divclk_divide = 2, and clkout0_divide_f = 35.0, calculated using the following equation:

$$F_{CLKOUTx} = F_{IN} \times \frac{\text{CLKFBOUT_MULT_F}}{\text{DIVCLK_DIVIDE} \times \text{CLKOUTx_DIVIDE_F}}$$

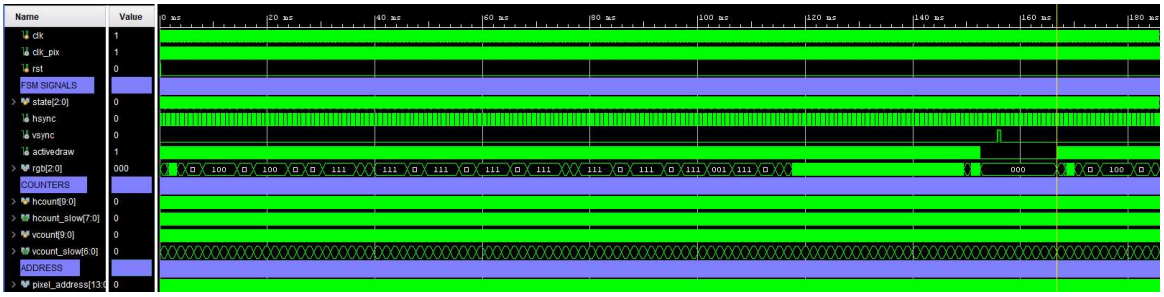
MMCM output-frequency equation.

The module generates the pixel address by concatenating the vertical and horizontal counters, {vcount, hcount}, and instantiates the VRAM and FSM modules.

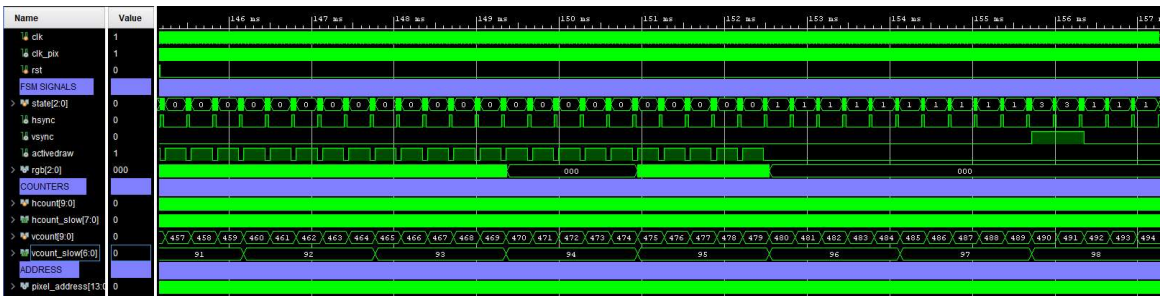
B.2 Verification

To simulate the counter and synchronization/active-video logic, a testbench named timing_and_color_driver_tb was developed. It initializes the clock and reset signals for a sufficiently long interval to count all pixels in the image.

One issue concerned the slow-clock timing: according to the waveforms, its frequency was exactly one-quarter of the FPGA board clock frequency. However, the 25.175 MHz clock period should be slightly shorter than four times the 100 MHz clock period. The issue was traced to the MMCM initialization parameters used to calculate clkout; some decimal values were not interpreted as intended. The parameters were subsequently set to appropriate values.



Timing simulation showing clocks, counters, and video signals (view 1).

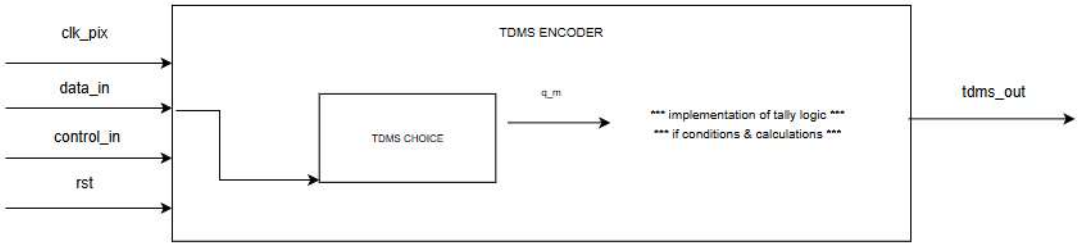


Timing simulation showing clocks, counters, and video signals (view 2).

PART C: TMDS Encoding

The aim of this part was to reduce noise and frequent transitions in the high-frequency bit stream using TMDS encoding.

C.1 Implementation



TMDS encoder architecture, including transition-minimization logic.

First, a transition-minimization module named tm_choice was implemented. It contains a combinational always block that counts the number of ones in the 8-bit input sequence. Depending on this count, the circuit selects either an XNOR or an XOR operation to produce a 9-bit output.

Case 1 (XOR): $y_n = x_n \oplus y_{n-1}$

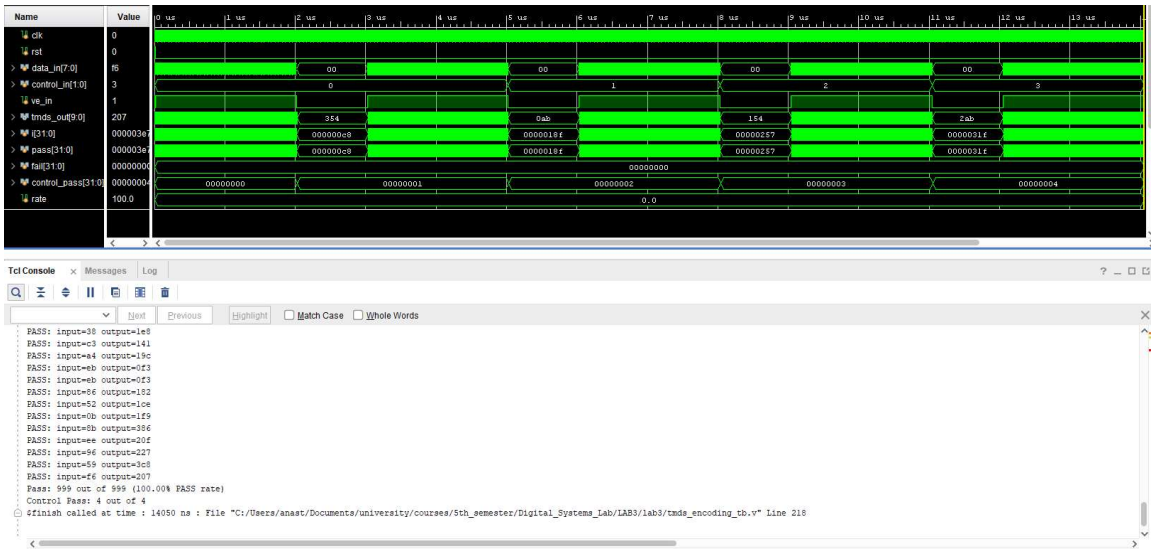
Case 2 (XNOR): $y_n = \neg(x_n \oplus y_{n-1})$

Next, the main tmds_encoder module was created. It uses a combinational always block to count the ones and zeros in the 9-bit sequence produced by tm_choice. A sequential always block compares the number of ones with the number of zeros. Based on this comparison, when ve_in (enable) is asserted, the encoder determines the signed 10-bit output relative to the 9-bit input and whether the running disparity counter (tally) should increase or decrease. The tally is kept close to zero, with values from −10 to 10. When ve_in is inactive, tally is reset to zero and tmds_out is selected according to the 2-bit control_in value:

- 2'b00: 10'b1101010100
- 2'b01: 10'b0010101011
- 2'b10: 10'b0101010100
- 2'b11: 10'b1010101011

C.2 Verification

The provided tmds_encoder_tb testbench was used to simulate and verify the TMDS encoding module. It applies a set of tmds_vectors and displays PASS or FAIL messages to confirm that each input is encoded correctly and produces the expected output.

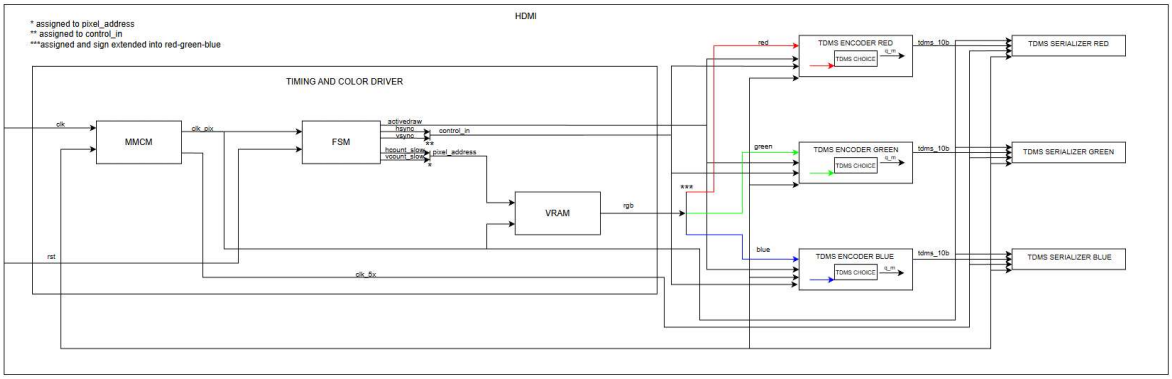


TMDS encoder testbench waveforms and simulation messages.

The only error encountered was using the ~ operator in the last tally-calculation condition to express the inversion of the most significant bit (MSB) of the 9-bit sequence. This was corrected by using the ! operator, which logically inverts a bit from 0 to 1 or from 1 to 0.

PART D: HDMI Controller Integration

D.1 Implementation



Integrated HDMI controller, including the TMDS encoders and serializers.

In this part, all the previously developed modules were connected to complete the transmission of bits over the HDMI link. The supplied tmds_serializer module was used to serialize the data, since each HDMI data lane transmits one bit at a time. A clock five times faster than the pixel clock was also generated by configuring the MMCM and choosing the appropriate coefficients for calculating clk_out using the following equation:

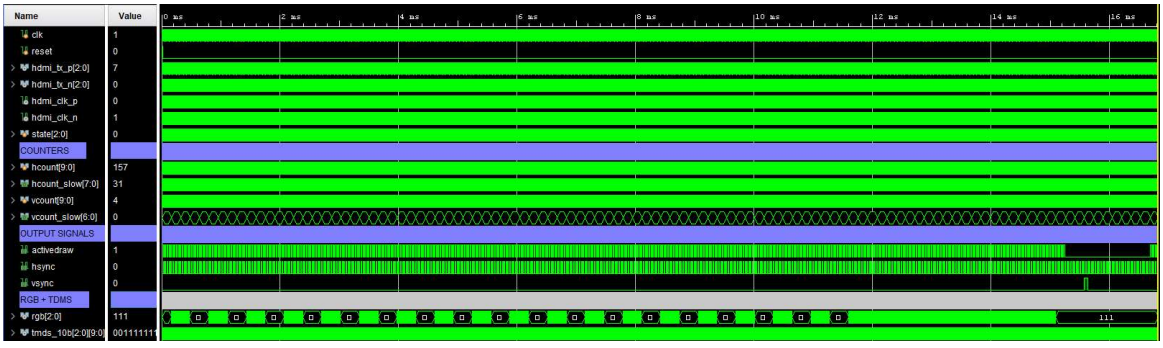
$$F_{CLKOUTx} = F_{IN} \times \frac{\text{CLKFBOUT_MULT_F}}{\text{DIVCLK_DIVIDE} \times \text{CLKOUTx_DIVIDE_F}}$$

MMCM frequency relation used to generate the serializer clock.

For the 8-bit encoders of the three color channels, a color value of 1 or 0 maps to 11111111 or 00000000, respectively. The module then produces one serial bit per color channel for transmission over the HDMI link.

D.2 Verification

To simulate the final integration and the overall operation of the HDMI link, a testbench named hdmi_tb was implemented. Its only function is to generate a clock with a period of 10 ns and assert and deassert reset for a suitable interval until the complete pixel sequence has been displayed.



Integrated HDMI timing and serialization simulation.

The waveforms show the following signals:

- Clocks: clk; clk_pixel, with a period approximately four times that of clk; and clk_5x, which runs five times faster than clk_pixel.
- Video timing signals: activedraw, hsync, vsync, and rgb.
- TMDS encoder outputs: tmds_10b[2], tmds_10b[1], and tmds_10b[0], which should change only on clk_pixel.

- Serializer output: tmds_signal[2:0], which should change on clk_5x (two bits per clk_5x cycle, one at each edge, as part of serialization).

D.3 Experiment and Final Implementation

During the practical implementation of the HDMI system on the FPGA board, no problems were encountered, and the initialized image appeared in full on the display. Timing proved correct, and hsync, vsync, and activedraw were asserted in their appropriate intervals. Correct memory storage, encoding, and serialization ensured that the pixels appeared in the intended order and that all functions operated as expected.

CONCLUSIONS

The project was completed successfully, implementing all the required components of an HDMI controller and addressing the challenges encountered during development. The greatest challenge was the correct initialization of the image in VRAM, particularly confusion regarding the storage order of data in memory (right to left) and handling each pixel color. All difficulties were resolved by identifying individual errors and thoroughly verifying every relevant module, both through simulation and through practical FPGA testing.